

GSOI 2026 夏季训练赛

第十一题解

Ehundategh & tfbz

Gioush 大队

2026 年 8 月

目录

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间

Contents

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间

数据点 1 ~ 5 25 Pts

- ▶ 直接枚举最终选择的样本集合，再按照原顺序模拟获取过程。只要某次获取后能量小于 0，这个集合就不合法。

数据点 1 ~ 5 25 Pts

- ▶ 直接枚举最终选择的样本集合，再按照原顺序模拟获取过程。只要某次获取后能量小于 0，这个集合就不合法。
- ▶ 枚举集合需要 2^n 次，检查一个集合需要 $\mathcal{O}(n)$ 的时间，总时间复杂度为 $\mathcal{O}(2^n n)$ 。

数据点 1 ~ 5 25 Pts

- ▶ 直接枚举最终选择的样本集合，再按照原顺序模拟获取过程。只要某次获取后能量小于 0，这个集合就不合法。
- ▶ 枚举集合需要 2^n 次，检查一个集合需要 $\mathcal{O}(n)$ 的时间，总时间复杂度为 $\mathcal{O}(2^n n)$ 。
- ▶ 这个做法保留了题目中最关键的限制：已经选中的样本仍然必须按照传送带顺序获取。接下来只需要避免重复检查相同的前缀。

数据点 6 ~ 10 50 Pts

- ▶ 定义 f_j 表示处理完当前前缀后，恰好获取 j 份样本时能够保留的最大能量。不可达状态记为负无穷，初始时 $f_0 = h$ 。

数据点 6 ~ 10 50 Pts

- ▶ 定义 f_j 表示处理完当前前缀后，恰好获取 j 份样本时能够保留的最大能量。不可达状态记为负无穷，初始时 $f_0 = h$ 。
- ▶ 处理 a_i 时倒序枚举 j 。若 $f_{j-1} + a_i \geq 0$ ，则有

$$f_j \leftarrow \max\{f_j, f_{j-1} + a_i\}.$$

数据点 6 ~ 10 50 Pts

- ▶ 定义 f_j 表示处理完当前前缀后，恰好获取 j 份样本时能够保留的最大能量。不可达状态记为负无穷，初始时 $f_0 = h$ 。
- ▶ 处理 a_i 时倒序枚举 j 。若 $f_{j-1} + a_i \geq 0$ ，则有

$$f_j \leftarrow \max\{f_j, f_{j-1} + a_i\}.$$

- ▶ 对相同的获取次数，保留能量更大的方案一定不劣。状态数为 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(n^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 11 ~ 15 75 Pts

- ▶ 特殊性质保证所有非负数都在负数之前。我们先获取全部非负样本，此后只剩下一段负数。

数据点 11 ~ 15 75 Pts

- ▶ 特殊性质保证所有非负数都在负数之前。我们先获取全部非负样本，此后只剩下一段负数。
- ▶ 对于负数部分，如果要获取相同数量的样本，优先选择数值较大的负数能够留下更多能量。因此将负数从大到小排序，依次获取到无法继续即可。

数据点 11 ~ 15 75 Pts

- ▶ 特殊性质保证所有非负数都在负数之前。我们先获取全部非负样本，此后只剩下一段负数。
- ▶ 对于负数部分，如果要获取相同数量的样本，优先选择数值较大的负数能够留下更多能量。因此将负数从大到小排序，依次获取到无法继续即可。
- ▶ 时间复杂度为 $\mathcal{O}(n \log n)$ 。这一档已经得到正解的核心交换原则：选择数量相同时，应当删掉最消耗能量的样本。

正解 100 Pts

- ▶ 从左到右处理样本，并暂时把每一份样本都加入当前方案。
用小根堆保存已加入样本的 a_i ，用 S 保存当前能量。

正解 100 Pts

- ▶ 从左到右处理样本，并暂时把每一份样本都加入当前方案。用小根堆保存已加入样本的 a_i ，用 S 保存当前能量。
- ▶ 如果加入 a_i 后 $S < 0$ ，当前选择无法全部保留。为了让选择数量只减少 1 且剩余能量最大，应当删去堆中最小的 a_i 。

正解 100 Pts

- ▶ 从左到右处理样本，并暂时把每一份样本都加入当前方案。用小根堆保存已加入样本的 a_i ，用 S 保存当前能量。
- ▶ 如果加入 a_i 后 $S < 0$ ，当前选择无法全部保留。为了让选择数量只减少 1 且剩余能量最大，应当删去堆中最小的 a_i 。
- ▶ 删除后能量增加 $-a_i$ 。因为加入前的能量非负，最小值不大于刚加入的数，所以一次删除一定足以恢复 $S \geq 0$ 。

正解 100 Pts

- ▶ 处理完任意前缀后，堆中保存一个合法方案。并且在选择数量相同的所有合法方案中，它的剩余能量最大。

正解 100 Pts

- ▶ 处理完任意前缀后，堆中保存一个合法方案。并且在选择数量相同的所有合法方案中，它的剩余能量最大。
- ▶ 若当前前缀无需删除，直接加入不会破坏这个性质。若必须删除一个数，删除最小值会使剩余和最大，因此也不会影响后续最优选择。

正解 100 Pts

- ▶ 处理完任意前缀后，堆中保存一个合法方案。并且在选择数量相同的所有合法方案中，它的剩余能量最大。
- ▶ 若当前前缀无需删除，直接加入不会破坏这个性质。若必须删除一个数，删除最小值会使剩余和最大，因此也不会影响后续最优选择。
- ▶ 最终堆的大小就是答案。时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- ▶ 处理完任意前缀后，堆中保存一个合法方案。并且在选择数量相同的所有合法方案中，它的剩余能量最大。
- ▶ 若当前前缀无需删除，直接加入不会破坏这个性质。若必须删除一个数，删除最小值会使剩余和最大，因此也不会影响后续最优选择。
- ▶ 最终堆的大小就是答案。时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ h 、前缀能量与 a_i 的累加结果都可能超过 `int`，实现时需要使用 `long long`。每组数据开始前还要清空堆。

正解 100 Pts

```
1  int T,n;
2  long long h,Line[MAXN];
3  priority_queue <long long,vector<long long>,greater<long long> > Q;
4  void Solve(){
5      scanf("%d%lld",&n,&h);
6      while(!Q.empty()) Q.pop();
7      for(int i=1;i<=n;i++) scanf("%lld",&Line[i]);
8      long long Sum=h;
9      for(int i=1;i<=n;i++){
10         Sum+=Line[i];Q.push(Line[i]);
11         if(Sum<0){
12             Sum-=Q.top();
13             Q.pop();
14         }
15     }
16     printf("%d\n",(int)Q.size());
```

Contents

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间

数据点 1 ~ 3 15 Pts

- ▶ 对每份方案直接建出新城市。依次以每座城市为起点搜索整棵树，可以求出新树中全部有序城市对的费用。

数据点 1 ~ 3 15 Pts

- ▶ 对每份方案直接建出新城市。依次以每座城市为起点搜索整棵树，可以求出新树中全部有序城市对的费用。
- ▶ 每份方案需要 $\mathcal{O}(n^2)$ 的时间，总时间复杂度为 $\mathcal{O}(qn^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 1 ~ 3 15 Pts

- ▶ 对每份方案直接建出新城市。依次以每座城市为起点搜索整棵树，可以求出新树中全部有序城市对的费用。
- ▶ 每份方案需要 $\mathcal{O}(n^2)$ 的时间，总时间复杂度为 $\mathcal{O}(qn^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 原树本身没有随方案改变。后续只需拆出真正与接入位置有关的那一部分费用。

数据点 4 ~ 7 35 Pts

- 记原树全部有序城市对的费用和为 S ，记城市 k 到全部原城市
的费用和为

$$D_k = \sum_{v=1}^n \text{cost}(k, v).$$

数据点 4 ~ 7 35 Pts

- 记原树全部有序城市对的费用和为 S ，记城市 k 到全部原城市的费用和为

$$D_k = \sum_{v=1}^n \text{cost}(k, v).$$

- 新城市接到 k ，新道路费用为 x 。从新城市到任意原城市 v 的费用为 $x + \text{cost}(k, v)$ 。

数据点 4 ~ 7 35 Pts

- ▶ 记原树全部有序城市对的费用和为 S ，记城市 k 到全部原城市的费用和为

$$D_k = \sum_{v=1}^n \text{cost}(k, v).$$

- ▶ 新城市接到 k ，新道路费用为 x 。从新城市到任意原城市 v 的费用为 $x + \text{cost}(k, v)$ 。
- ▶ 反向费用相同，所以一份方案的答案为

$$S + 2(D_k + nx).$$

数据点 4 ~ 7 35 Pts

- ▶ 任选城市 1 为根。设 v 是 u 的儿子，边 (u, v) 的费用为 w ，则必须经过这条边的有序城市对数量为

$$2 \text{Size}(v)(n - \text{Size}(v))$$

数据点 4 ~ 7 35 Pts

- ▶ 任选城市 1 为根。设 v 是 u 的儿子，边 (u, v) 的费用为 w ，则必须经过这条边的有序城市对数量为

$$2 \operatorname{Size}(v)(n - \operatorname{Size}(v))$$

- ▶ 因此

$$S = \sum_{(u,v)} 2 \operatorname{Size}(v)(n - \operatorname{Size}(v))w_{u,v}.$$

一次搜索即可求出 S 。每份方案再从 k 出发搜索，求出 D_k 。

数据点 4 ~ 7 35 Pts

- ▶ 任选城市 1 为根。设 v 是 u 的儿子，边 (u, v) 的费用为 w ，则必须经过这条边的有序城市对数量为

$$2 \operatorname{Size}(v)(n - \operatorname{Size}(v))$$

- ▶ 因此

$$S = \sum_{(u,v)} 2 \operatorname{Size}(v)(n - \operatorname{Size}(v))w_{u,v}.$$

一次搜索即可求出 S 。每份方案再从 k 出发搜索，求出 D_k 。

- ▶ 总时间复杂度为 $\mathcal{O}(nq + n)$ 。瓶颈已经只剩下重复计算不同的 D_k 。

数据点 8 ~ 11 55 Pts

- 特殊性质保证所有方案都接入城市 1，因此只需预处理一次 D_1 。

数据点 8 ~ 11 55 Pts

- ▶ 特殊性质保证所有方案都接入城市 1，因此只需预处理一次 D_1 。
- ▶ 搜索时记录城市 1 到当前城市的费用并累加，即可得到

$$D_1 = \sum_{v=1}^n \text{cost}(1, v).$$

同一次搜索仍然按边的贡献计算 S 。

数据点 8 ~ 11 55 Pts

- ▶ 特殊性质保证所有方案都接入城市 1，因此只需预处理一次 D_1 。
- ▶ 搜索时记录城市 1 到当前城市的费用并累加，即可得到

$$D_1 = \sum_{v=1}^n \text{cost}(1, v).$$

同一次搜索仍然按边的贡献计算 S 。

- ▶ 每份方案直接代入 $S + 2(D_1 + nx)$ 。总时间复杂度为 $\mathcal{O}(n + q)$ 。

数据点 12 ~ 15 75 Pts

- 树是一条链。令 $p_i = \text{cost}(1, i)$ ，再令 $P_i = \sum_{j=1}^i p_j$ 。对于接入位置 k ，

$$\begin{aligned} D_k &= (k-1)p_k - P_{k-1} \\ &\quad + (P_n - P_k) - (n-k)p_k. \end{aligned}$$

数据点 12 ~ 15 75 Pts

- 树是一条链。令 $p_i = \text{cost}(1, i)$ ，再令 $P_i = \sum_{j=1}^i p_j$ 。对于接入位置 k ，

$$D_k = (k-1)p_k - P_{k-1} \\ + (P_n - P_k) - (n-k)p_k.$$

- 式子的两行分别统计 k 左侧和右侧的费用和，所以可以在 $\mathcal{O}(1)$ 时间求出 D_k 。

数据点 12 ~ 15 75 Pts

- ▶ 树是一条链。令 $p_i = \text{cost}(1, i)$ ，再令 $P_i = \sum_{j=1}^i p_j$ 。对于接入位置 k ，

$$D_k = (k-1)p_k - P_{k-1} \\ + (P_n - P_k) - (n-k)p_k.$$

- ▶ 式子的两行分别统计 k 左侧和右侧的费用和，所以可以在 $\mathcal{O}(1)$ 时间求出 D_k 。
- ▶ 总时间复杂度为 $\mathcal{O}(n+q)$ 。链上相邻位置的费用和变化，正是树上换根方程的雏形。

正解 100 Pts

- 固定根为 1。令 $\text{Size}(u)$ 表示 u 的子树大小，令 F_u 表示 u 到其子树内全部城市的费用和。

正解 100 Pts

- ▶ 固定根为 1。令 $\text{Size}(u)$ 表示 u 的子树大小，令 F_u 表示 u 到其子树内全部城市的费用和。
- ▶ 若 v 枚举 u 的儿子，换根前的子树 DP 方程为

$$\text{Size}(u) = 1 + \sum_v \text{Size}(v),$$

$$F_u = \sum_v (F_v + \text{Size}(v)w_{u,v}).$$

正解 100 Pts

- ▶ 固定根为 1。令 $\text{Size}(u)$ 表示 u 的子树大小，令 F_u 表示 u 到其子树内全部城市的费用和。
- ▶ 若 v 枚举 u 的儿子，换根前的子树 DP 方程为

$$\text{Size}(u) = 1 + \sum_v \text{Size}(v),$$

$$F_u = \sum_v (F_v + \text{Size}(v)w_{u,v}).$$

- ▶ 因此 $D_1 = F_1$ 。实现时也可以在搜索中直接传入根到当前城市的费用，再把它们全部累加。

正解 100 Pts

- 接下来令 $D_u = \sum_{x=1}^n \text{cost}(u, x)$ 。设 v 是 u 的儿子，边 (u, v) 的费用为 w 。

正解 100 Pts

- ▶ 接下来令 $D_u = \sum_{x=1}^n \text{cost}(u, x)$ 。设 v 是 u 的儿子，边 (u, v) 的费用为 w 。
- ▶ 把根从 u 换到 v 后， v 子树内的 $\text{Size}(v)$ 个城市费用都减少 w ，其余 $n - \text{Size}(v)$ 个城市费用都增加 w 。

正解 100 Pts

- ▶ 接下来令 $D_u = \sum_{x=1}^n \text{cost}(u, x)$ 。设 v 是 u 的儿子，边 (u, v) 的费用为 w 。
- ▶ 把根从 u 换到 v 后， v 子树内的 $\text{Size}(v)$ 个城市费用都减少 w ，其余 $n - \text{Size}(v)$ 个城市费用都增加 w 。
- ▶ 所以完整的换根 DP 方程为

$$\begin{aligned} D_v &= D_u - \text{Size}(v)w + (n - \text{Size}(v))w \\ &= D_u + (n - 2\text{Size}(v))w. \end{aligned}$$

正解 100 Pts

- 第二次搜索沿树应用换根方程，即可求出全部 D_u 。

正解 100 Pts

- ▶ 第二次搜索沿树应用换根方程，即可求出全部 D_u 。
- ▶ 原树的有序城市对费用和为

$$S = \sum_{u=1}^n D_u.$$

正解 100 Pts

- ▶ 第二次搜索沿树应用换根方程，即可求出全部 D_u 。
- ▶ 原树的有序城市对费用和为

$$S = \sum_{u=1}^n D_u.$$

- ▶ 这里每个有序对 (u, v) 恰好在 D_u 中出现一次，所以既没有遗漏，也没有重复。每份方案最终输出

$$S + 2(D_k + nx).$$

正解 100 Pts

- ▶ 预处理时间复杂度为 $\mathcal{O}(n)$ ，每次询问为 $\mathcal{O}(1)$ ，总时间复杂度为 $\mathcal{O}(n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- ▶ 预处理时间复杂度为 $\mathcal{O}(n)$ ，每次询问为 $\mathcal{O}(1)$ ，总时间复杂度为 $\mathcal{O}(n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 所有计算都在模 998,244,353 意义下进行。 $n - 2 \text{Size}(v)$ 可能为负数，完成乘法与加法后要把结果重新规范到 $[0, 998,244,353)$ 。

正解 100 Pts

- ▶ 预处理时间复杂度为 $\mathcal{O}(n)$ ，每次询问为 $\mathcal{O}(1)$ ，总时间复杂度为 $\mathcal{O}(n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 所有计算都在模 998,244,353 意义下进行。 $n - 2 \text{Size}(v)$ 可能为负数，完成乘法与加法后要把结果重新规范到 $[0, 998,244,353)$ 。
- ▶ 多组数据之间需要清空链式前向星表头、Size、 D 与边计数。代码中的 Count 对应这里的 D 。

正解 100 Pts

```
1 void DFS_F(int Now,int From,int Value){
2     Size[Now]=1;
3     for(int i=Head[Now];i;i=Edge[i].Next){
4         int To=Edge[i].Ed;
5         if(To==From) continue;
6         DFS_F(To,Now,Calc(Value,Edge[i].Value));
7         Size[Now]+=Size[To];
8         Count[Root]=Calc(Count[Root],Calc(Value,Edge[i].Value));
9     }
10 }
```

正解 100 Pts

```
1 void DFS_Pass(int Now,int From){
2     for(int i=Head[Now];i;i=Edge[i].Next){
3         int To=Edge[i].Ed;
4         if(To==From) continue;
5         Count[To]=Calc(Count[Now],Mul((n-Size[To]-Size[To]),Edge[i].
        Value));
6         Count[To]=(Count[To]%Mod+Mod)%Mod;
7         DFS_Pass(To,Now);
8     }
9 }
10 void Build(int Line,int Cost){
11     printf("%d\n",Calc(Sum,Mul(2,Calc(Count[Line],Mul(Cost,n)))));
12 }
```

正解 100 Pts

```
1 void Solve(){
2     int Q;
3     scanf("%d%d",&n,&Q);
4     memset(Head,0,sizeof(int)*(n+2));
5     memset(Count,0,sizeof(int)*(n+2));
6     memset(Size,0,sizeof(int)*(n+2));
7     Total=Sum=0;Root=1;
8     int In1,In2,In3;
9     for(int i=1;i<=n-1;i++){
10         scanf("%d%d%d",&In1,&In2,&In3);
11         Edge_Add(In1,In2,In3);
```

正解 100 Pts

```
1      Edge_Add(In2, In1, In3);  
2  }  
3  DFS_F(Root, Root, 0);  
4  DFS_Pass(Root, Root);  
5  for(int i=1; i<=n; i++) Sum=Calc(Sum, Count[i]);  
6  while(Q--){  
7      scanf("%d%d", &In1, &In2);  
8      Build(In1, In2);  
9  }
```

Contents

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间

数据点 1 ~ 5 20 Pts

- ▶ 对每次询问扫描区间 $[l, r]$ ，把出现过的颜色加入集合，集合大小就是答案。

数据点 1 ~ 5 20 Pts

- ▶ 对每次询问扫描区间 $[l, r]$ ，把出现过的颜色加入集合，集合大小就是答案。
- ▶ 使用平衡树集合时，时间复杂度为 $\mathcal{O}(nq \log n)$ 。这一做法直接按照“不同颜色”的定义统计。

数据点 1 ~ 5 20 Pts

- ▶ 对每次询问扫描区间 $[l, r]$ ，把出现过的颜色加入集合，集合大小就是答案。
- ▶ 使用平衡树集合时，时间复杂度为 $\mathcal{O}(nq \log n)$ 。这一做法直接按照“不同颜色”的定义统计。
- ▶ 同一颜色在一次询问中只需要贡献一次，因此我们真正需要维护的是每种颜色选出的一个代表位置。

数据点 6 ~ 10 40 Pts

- ▶ 使用时间戳数组代替集合。扫描询问区间时，若颜色 a_i 的时间戳不是当前询问编号，就把答案加一并更新时间戳。

数据点 6 ~ 10 40 Pts

- ▶ 使用时间戳数组代替集合。扫描询问区间时，若颜色 a_i 的时间戳不是当前询问编号，就把答案加一并更新时间戳。
- ▶ 每个位置只进行常数操作，总时间复杂度降为 $\mathcal{O}(nq)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 6 ~ 10 40 Pts

- ▶ 使用时间戳数组代替集合。扫描询问区间时，若颜色 a_i 的时间戳不是当前询问编号，就把答案加一并更新时间戳。
- ▶ 每个位置只进行常数操作，总时间复杂度降为 $\mathcal{O}(nq)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 瓶颈仍然是不同询问反复扫描相同前缀。接下来让询问端点具有顺序。

数据点 11 ~ 14 56 Pts

- ▶ 所有询问左端点都是 1。顺序扫描序列，维护前缀中已经出现过的颜色数量。

数据点 11 ~ 14 56 Pts

- ▶ 所有询问左端点都是 1。顺序扫描序列，维护前缀中已经出现过的颜色数量。
- ▶ 第一次遇到颜色 a_i 时把计数加一，询问 $[1, r]$ 的答案就是扫描到 r 时的计数。

数据点 11 ~ 14 56 Pts

- ▶ 所有询问左端点都是 1。顺序扫描序列，维护前缀中已经出现过的颜色数量。
- ▶ 第一次遇到颜色 a_i 时把计数加一，询问 $[1, r]$ 的答案就是扫描到 r 时的计数。
- ▶ 总时间复杂度为 $\mathcal{O}(n + q)$ 。这一档说明按照右端点推进时，可以持续复用之前加入的元素。

数据点 15 ~ 19 76 Pts

- ▶ 询问的右端点单调不降。处理到右端点 r 时，只保留每种颜色在前缀 $[1, r]$ 中最后一次出现的位置。

数据点 15 ~ 19 76 Pts

- ▶ 询问的右端点单调不降。处理到右端点 r 时，只保留每种颜色在前缀 $[1, r]$ 中最后一次出现的位置。
- ▶ 这些代表位置上放置 1，其余位置放置 0。于是区间 $[l, r]$ 的不同颜色数就是这个区间的权值和。

数据点 15 ~ 19 76 Pts

- ▶ 询问的右端点单调不降。处理到右端点 r 时，只保留每种颜色在前缀 $[1, r]$ 中最后一次出现的位置。
- ▶ 这些代表位置上放置 1，其余位置放置 0。于是区间 $[l, r]$ 的不同颜色数就是这个区间的权值和。
- ▶ 用树状数组维护单点修改和区间和，总时间复杂度为 $\mathcal{O}((n + q) \log n)$ 。

正解 100 Pts

- ▶ 一般询问的右端点没有顺序。把询问按 r 从小到大排序，再沿用上一档的维护过程。

正解 100 Pts

- ▶ 一般询问的右端点没有顺序。把询问按 r 从小到大排序，再沿用上一档的维护过程。
- ▶ 加入位置 i 时，若颜色 a_i 上一次出现在 $\text{Last}(a_i)$ ，先在旧位置减一，再在位置 i 加一，最后令 $\text{Last}(a_i) = i$ 。

正解 100 Pts

- ▶ 一般询问的右端点没有顺序。把询问按 r 从小到大排序，再沿用上一档的维护过程。
- ▶ 加入位置 i 时，若颜色 a_i 上一次出现在 $\text{Last}(a_i)$ ，先在旧位置减一，再在位置 i 加一，最后令 $\text{Last}(a_i) = i$ 。
- ▶ 此时树状数组中每种已经出现的颜色恰好有一个 1，而且这个 1 位于其最后出现位置。

正解 100 Pts

- ▶ 对询问 $[l, r]$, 某种颜色在区间中出现, 当且仅当它在前缀 $[1, r]$ 中的最后出现位置不小于 l 。

正解 100 Pts

- ▶ 对询问 $[l, r]$, 某种颜色在区间中出现, 当且仅当它在前缀 $[1, r]$ 中的最后出现位置不小于 l 。
- ▶ 因此答案为

$$\text{Query}(r) - \text{Query}(l - 1).$$

正解 100 Pts

- ▶ 对询问 $[l, r]$, 某种颜色在区间中出现, 当且仅当它在前缀 $[1, r]$ 中的最后出现位置不小于 l 。
- ▶ 因此答案为
$$\text{Query}(r) - \text{Query}(l - 1).$$
- ▶ 排序会打乱询问顺序, 所以记录原编号 p , 把答案写入 $\text{Ans}(p)$, 最后按编号输出。

正解 100 Pts

- 时间复杂度为 $\mathcal{O}((n + q) \log n)$ ，空间复杂度为 $\mathcal{O}(n + q)$ 。

正解 100 Pts

- ▶ 时间复杂度为 $\mathcal{O}((n + q) \log n)$ ，空间复杂度为 $\mathcal{O}(n + q)$ 。
- ▶ 多组数据中，树状数组只需要清空前 n 个位置。颜色最后出现位置可以在输出后枚举本组序列并清零，避免每次清空完整值域。

正解 100 Pts

- ▶ 时间复杂度为 $\mathcal{O}((n + q) \log n)$ ，空间复杂度为 $\mathcal{O}(n + q)$ 。
- ▶ 多组数据中，树状数组只需要清空前 n 个位置。颜色最后出现位置可以在输出后枚举本组序列并清零，避免每次清空完整值域。
- ▶ 右端点相同时，询问之间的次序不会影响答案。排序比较函数只需要先比较 r ，再比较 l 。

正解 100 Pts

```
1  int T,n,q,Line[MAXN],Last[MAXN],C[MAXN],Ans[MAXN];
2  struct query{
3      int l,r,p;
4  }Q[MAXN];
5  bool cmp(query a,query b){
6      if(a.r!=b.r) return a.r<b.r;
7      return a.l<b.l;
8  }
9  int lowbit(int x){return x&-x;}
10 void Modify(int x,int Val){
11     for(;x<=n;x+=lowbit(x)) C[x]+=Val;
12 }
13 int Query(int x){
14     int Ret=0;
15     for(;x;x-=lowbit(x)) Ret+=C[x];
16     return Ret;
17 }
```

正解 100 Pts

```
1 void Solve(){
2     scanf("%d%d",&n,&q);
3     memset(C,0,sizeof(C));
4     for(int i=1;i<=n;i++) scanf("%d",&Line[i]);
5     for(int i=1;i<=q;i++){
6         scanf("%d%d",&Q[i].l,&Q[i].r);
7         Q[i].p=i;
8     }
9     sort(Q+1,Q+q+1,cmp);
10    int Now=0;
11    for(int i=1;i<=q;i++){
```

正解 100 Pts

```
1      while(Now<Q[i].r){
2          Now++;
3          if(Last[Line[Now]]) Modify(Last[Line[Now]],-1);
4          Modify(Now,1);
5          Last[Line[Now]]=Now;
6      }
7      Ans[Q[i].p]=Query(Q[i].r)-Query(Q[i].l-1);
8  }
9  for(int i=1;i<=q;i++) printf("%d\n",Ans[i]);
10 for(int i=1;i<=n;i++) Last[Line[i]]=0;
11 }
```

Contents

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间

数据点 1 ~ 5 20 Pts

- ▶ 二分完成任务的时间。对于固定时限，枚举每名特工最终潜伏的据点，共有至多 $(n-1)^m$ 种方案。

数据点 1 ~ 5 20 Pts

- ▶ 二分完成任务的时间。对于固定时限，枚举每名特工最终潜伏的据点，共有至多 $(n-1)^m$ 种方案。
- ▶ 预处理任意两点间的距离。枚举到一份方案后，先检查每名特工能否在时限内到达对应据点，再检查每条根到叶路径上是否至少有一个潜伏据点。

数据点 1 ~ 5 20 Pts

- ▶ 二分完成任务的时间。对于固定时限，枚举每名特工最终潜伏的据点，共有至多 $(n-1)^m$ 种方案。
- ▶ 预处理任意两点间的距离。枚举到一份方案后，先检查每名特工能否在时限内到达对应据点，再检查每条根到叶路径上是否至少有一个潜伏据点。
- ▶ 判定的时间复杂度为 $\mathcal{O}((n-1)^m(n+m))$ 。这一档说明固定时间后只需处理两件事：特工的可达性与叶子路径的覆盖。

数据点 6 ~ 10 40 Pts

- ▶ 叶子数 $L \leq 10$ 。给叶子编号，定义 C_u 为 u 的子树中叶子编号构成的二进制集合。特工潜伏在 u ，恰好能够截断 C_u 中的全部叶子。

数据点 6 ~ 10 40 Pts

- ▶ 叶子数 $L \leq 10$ 。给叶子编号，定义 C_u 为 u 的子树中叶子编号构成的二进制集合。特工潜伏在 u ，恰好能够截断 C_u 中的全部叶子。
- ▶ 固定时限后，若 $\text{dist}(\text{Army}_i, u)$ 不超过时限，就可以让第 i 名特工贡献集合 C_u 。

数据点 6 ~ 10 40 Pts

- ▶ 叶子数 $L \leq 10$ 。给叶子编号，定义 C_u 为 u 的子树中叶子编号构成的二进制集合。特工潜伏在 u ，恰好能够截断 C_u 中的全部叶子。
- ▶ 固定时限后，若 $\text{dist}(\text{Army}_i, u)$ 不超过时限，就可以让第 i 名特工贡献集合 C_u 。
- ▶ 定义 $f_{i,S}$ 表示处理完前 i 名特工后，能否覆盖叶子集合 S 。初始状态为 $f_{0,0} = 1$ ，其余状态均为 0。

数据点 6 ~ 10 40 Pts

- 可以不使用第 i 名特工，也可以枚举它能够到达的据点 u ，于是转移为

$$f_{i,S} \leftarrow f_{i,S} \vee f_{i-1,S},$$

$$f_{i,S \cup C_u} \leftarrow f_{i,S \cup C_u} \vee f_{i-1,S}.$$

数据点 6 ~ 10 40 Pts

- 可以不使用第 i 名特工，也可以枚举它能够到达的据点 u ，于是转移为

$$f_{i,S} \leftarrow f_{i,S} \vee f_{i-1,S},$$

$$f_{i,SUC_u} \leftarrow f_{i,SUC_u} \vee f_{i-1,S}.$$

- 全部叶子对应的状态可达，当且仅当当前时限可行。这个判定完整保留了上一档的“每名特工选择一个据点”，只把覆盖结果压成了叶子集合。

数据点 6 ~ 10 40 Pts

- 可以不使用第 i 名特工，也可以枚举它能够到达的据点 u ，于是转移为

$$\begin{aligned}f_{i,S} &\leftarrow f_{i,S} \vee f_{i-1,S}, \\f_{i,SUC_u} &\leftarrow f_{i,SUC_u} \vee f_{i-1,S}.\end{aligned}$$

- 全部叶子对应的状态可达，当且仅当当前时限可行。这个判定完整保留了上一档的“每名特工选择一个据点”，只把覆盖结果压成了叶子集合。
- 预处理任意两点距离为 $\mathcal{O}(n^3)$ ，总时间复杂度为 $\mathcal{O}(n^3 + \log V \cdot mn2^L)$ ，空间复杂度为 $\mathcal{O}(n^2 + 2^L)$ 。

数据点 11 ~ 14 56 Pts

- ▶ 根节点只有一个儿子。固定时限后，每名特工尽量向上移动，并停在根节点下方能够到达的最高据点。

数据点 11 ~ 14 56 Pts

- ▶ 根节点只有一个儿子。固定时限后，每名特工尽量向上移动，并停在根节点下方能够到达的最高据点。
- ▶ 若特工能够到达根节点，就让它停在根的唯一儿子处。否则标记它最终到达的据点。若一个据点被标记，整棵子树都已截断；否则只有它不是叶子并且所有儿子都被覆盖时，这棵子树才被覆盖。

数据点 11 ~ 14 56 Pts

- ▶ 根节点只有一个儿子。固定时限后，每名特工尽量向上移动，并停在根节点下方能够到达的最高据点。
- ▶ 若特工能够到达根节点，就让它停在根的唯一儿子处。否则标记它最终到达的据点。若一个据点被标记，整棵子树都已截断；否则只有它不是叶子并且所有儿子都被覆盖时，这棵子树才被覆盖。
- ▶ 这一档不存在根分支之间的调度。倍增移动与一次树上覆盖检查即可完成判定，单次判定复杂度为 $\mathcal{O}((n+m)\log n)$ 。

数据点 15 ~ 19 76 Pts

- ▶ 树是一棵菊花图。令根到叶子 u 的边长为 w_u 。若 $\text{Limit} < w_u$ ，位于 u 的特工无法到达根节点，留在原地便能覆盖分支 u 。

数据点 15 ~ 19 76 Pts

- ▶ 树是一棵菊花图。令根到叶子 u 的边长为 w_u 。若 $\text{Limit} < w_u$ ，位于 u 的特工无法到达根节点，留在原地便能覆盖分支 u 。
- ▶ 否则特工到达根节点后的剩余时间为 $\text{Rest} = \text{Limit} - w_u$ 。若 $\text{Rest} < w_u$ 且分支 u 还没有其他保障，应当先保留一名特工在原分支。

数据点 15 ~ 19 76 Pts

- ▶ 树是一棵菊花图。令根到叶子 u 的边长为 w_u 。若 $\text{Limit} < w_u$ ，位于 u 的特工无法到达根节点，留在原地便能覆盖分支 u 。
- ▶ 否则特工到达根节点后的剩余时间为 $\text{Rest} = \text{Limit} - w_u$ 。若 $\text{Rest} < w_u$ 且分支 u 还没有其他保障，应当先保留一名特工在原分支。
- ▶ 同一分支需要保留时，保留 Rest 最小的特工一定不劣，这样可以把能力更强的特工交给其他分支。

数据点 15 ~ 19 76 Pts

- 完成预留后，剩余特工只能从根节点进入某个尚未覆盖的分支。剩余时间为 x 的特工能够进入分支 u ，当且仅当 $x \geq w_u$ 。

数据点 15 ~ 19 76 Pts

- ▶ 完成预留后，剩余特工只能从根节点进入某个尚未覆盖的分支。剩余时间为 x 的特工能够进入分支 u ，当且仅当 $x \geq w_u$ 。
- ▶ 将全部剩余时间与全部未覆盖分支的边长分别从小到大排序。依次尝试满足当前最小需求，过小的特工直接跳过。

数据点 15 ~ 19 76 Pts

- ▶ 完成预留后，剩余特工只能从根节点进入某个尚未覆盖的分支。剩余时间为 x 的特工能够进入分支 u ，当且仅当 $x \geq w_u$ 。
- ▶ 将全部剩余时间与全部未覆盖分支的边长分别从小到大排序。依次尝试满足当前最小需求，过小的特工直接跳过。
- ▶ 若某名特工连最小需求都不能满足，它也不能满足后续需求；若它能够满足最小需求，把它用于这里不会影响更强特工完成更大需求。于是贪心正确。

正解 100 Pts

- ▶ 若时间 t 可以完成任务，那么任何更大的时间也可以完成，因此答案具有单调性，可以二分。

正解 100 Pts

- ▶ 若时间 t 可以完成任务，那么任何更大的时间也可以完成，因此答案具有单调性，可以二分。
- ▶ 先用倍增预处理每个据点的 2^j 级祖先，以及向上跳到该祖先需要的距离。判定 Limit 时，每名特工尽量向上跳，但先不越过根节点。

正解 100 Pts

- ▶ 若时间 t 可以完成任务，那么任何更大的时间也可以完成，因此答案具有单调性，可以二分。
- ▶ 先用倍增预处理每个据点的 2^j 级祖先，以及向上跳到该祖先需要的距离。判定 Limit 时，每名特工尽量向上跳，但先不越过根节点。
- ▶ 若特工无法到达根节点，就标记它在时限内能够到达的最高据点。它无法进入其他根分支，停在这里可以覆盖尽量大的原子树，所以一定不劣。

正解 100 Pts

- 定义 $\text{Cover}(u)$ 表示 u 的子树内所有叶子路径是否已经被这些局部特工截断。完整递推为

$$\text{Cover}(u) = \text{Mark}(u) \vee \left(u \text{ 不是叶子} \wedge \bigwedge_{v \in \text{son}(u)} \text{Cover}(v) \right).$$

正解 100 Pts

- 定义 $\text{Cover}(u)$ 表示 u 的子树内所有叶子路径是否已经被这些局部特工截断。完整递推为

$$\text{Cover}(u) = \text{Mark}(u) \vee \left(u \text{ 不是叶子} \wedge \bigwedge_{v \in \text{son}(u)} \text{Cover}(v) \right).$$

- 若 u 被标记，所有经过 u 的路径都已截断；否则必须让每个儿子子树分别被截断。这同时说明了递推的充分性与必要性。

正解 100 Pts

- ▶ 定义 $\text{Cover}(u)$ 表示 u 的子树内所有叶子路径是否已经被这些局部特工截断。完整递推为

$$\text{Cover}(u) = \text{Mark}(u) \vee \left(u \text{ 不是叶子} \wedge \bigwedge_{v \in \text{son}(u)} \text{Cover}(v) \right).$$

- ▶ 若 u 被标记, 所有经过 u 的路径都已截断; 否则必须让每个儿子子树分别被截断。这同时说明了递推的充分性与必要性。
- ▶ 对根的每个儿子分别检查, 把尚未覆盖的根儿子记为需要外部特工的分支。

正解 100 Pts

- 若一名特工能够到达根节点，记它原来所在的根分支为 r ，到达根节点后的剩余时间为

$$\text{Rest} = \text{Limit} - \text{dist}(\text{Army}, 1).$$

正解 100 Pts

- ▶ 若一名特工能够到达根节点，记它原来所在的根分支为 r ，到达根节点后的剩余时间为

$$\text{Rest} = \text{Limit} - \text{dist}(\text{Army}, 1).$$

- ▶ 它从根节点进入根儿子 v ，当且仅当 $\text{Rest} \geq w(1, v)$ 。所以这名特工只需记录二元组 (Rest, r) 。

正解 100 Pts

- ▶ 若一名特工能够到达根节点，记它原来所在的根分支为 r ，到达根节点后的剩余时间为

$$\text{Rest} = \text{Limit} - \text{dist}(\text{Army}, 1).$$

- ▶ 它从根节点进入根儿子 v ，当且仅当 $\text{Rest} \geq w(1, v)$ 。所以这名特工只需记录二元组 (Rest, r) 。
- ▶ 对于已经完成局部覆盖后的树，全部剩余信息就是这些二元组，以及每个未覆盖分支的根边长度。

正解 100 Pts

- ▶ 不能立刻把所有特工投入统一匹配。若分支 r 尚未覆盖，并且来自 r 的最小 Rest 满足 $\text{Rest} < w(1, r)$ ，就先让这名特工留在根儿子 r 。

正解 100 Pts

- ▶ 不能立刻把所有特工投入统一匹配。若分支 r 尚未覆盖，并且来自 r 的最小 Rest 满足 $\text{Rest} < w(1, r)$ ，就先让这名特工留在根儿子 r 。
- ▶ 假设某个可行方案没有这样做，那么分支 r 必须由另一名剩余时间至少为 $w(1, r)$ 的特工负责。交换两名特工后，较弱特工留在原分支，较强特工接替它原来的任务，方案仍然可行。

正解 100 Pts

- ▶ 不能立刻把所有特工投入统一匹配。若分支 r 尚未覆盖，并且来自 r 的最小 Rest 满足 $\text{Rest} < w(1, r)$ ，就先让这名特工留在根儿子 r 。
- ▶ 假设某个可行方案没有这样做，那么分支 r 必须由另一名剩余时间至少为 $w(1, r)$ 的特工负责。交换两名特工后，较弱特工留在原分支，较强特工接替它原来的任务，方案仍然可行。
- ▶ 因而总能得到一个满足上述预留规则的最优方案。选择同分支中最弱的特工预留，可以让剩余匹配能力最大。

正解 100 Pts

- ▶ 完成预留后，把剩余时间从小到大排序，把仍未覆盖分支的根边长度也从小到大排序。

正解 100 Pts

- ▶ 完成预留后，把剩余时间从小到大排序，把仍未覆盖分支的根边长度也从小到大排序。
- ▶ 若当前特工不能满足最小需求，它也不可能满足后面的更大需求，可以直接跳过；否则把它用于最小需求。

正解 100 Pts

- ▶ 完成预留后，把剩余时间从小到大排序，把仍未覆盖分支的根边长度也从小到大排序。
- ▶ 若当前特工不能满足最小需求，它也不可能满足后面的更大需求，可以直接跳过；否则把它用于最小需求。
- ▶ 在任意可行匹配中，与最小需求匹配的特工不会弱于当前选择。交换这两名特工后，其余匹配仍然成立，所以双指针贪心是充要的。

正解 100 Pts

- ▶ 令 V 为全部边长之和。任意两点间的简单路径长度都不超过 V ，所以若存在可行安排，时间 V 一定已经足够。若此时仍不可行，就输出 Pity!。

正解 100 Pts

- ▶ 令 V 为全部边长之和。任意两点间的简单路径长度都不超过 V ，所以若存在可行安排，时间 V 一定已经足够。若此时仍不可行，就输出 Pity!。
- ▶ 一次判定包含倍增移动、树上覆盖和排序，时间复杂度为 $\mathcal{O}((n+m)\log n)$ 。

正解 100 Pts

- ▶ 令 V 为全部边长之和。任意两点间的简单路径长度都不超过 V ，所以若存在可行安排，时间 V 一定已经足够。若此时仍不可行，就输出 Pity!。
- ▶ 一次判定包含倍增移动、树上覆盖和排序，时间复杂度为 $\mathcal{O}((n+m)\log n)$ 。
- ▶ 总时间复杂度为 $\mathcal{O}((n+m)\log n \log V)$ ，空间复杂度为 $\mathcal{O}(n \log n)$ 。路径长度、剩余时间与 V 都需要使用 long long。

正解 100 Pts

```
1  for(int i=1;i<=m;i++){
2      int Now=Army[i];
3      long long UsedValue=0;
4      for(int j=17;j>=0;j--){
5          if(Fa[Now][j]>1&&UsedValue+Value[Now][j]<=Limit){
6              UsedValue+=Value[Now][j];
7              Now=Fa[Now][j];
8          }
9      }
10     if(Fa[Now][0]==1&&UsedValue+Value[Now][0]<=Limit){
11         Free[++cf]={Limit-UsedValue-Value[Now][0],Now};
12     }
13     else Mark[Now]=true;
14 }
15 for(int i=Head[1];i;i=Edge[i].Next){
16     int To=Edge[i].Ed;
17     if(!Cover(To,1)) Need[To]=true;
18 }
```

正解 100 Pts

```
1  sort(Free+1,Free+cf+1,cmp);
2  for(int i=1;i<=cf;i++){
3      int Root=Free[i].Root;
4      if(!MinID[Root]) MinID[Root]=i;
5  }
6  for(int i=Head[1];i;i=Edge[i].Next){
7      int To=Edge[i].Ed;
8      if(!Need[To]) continue;
9      int Pos=MinID[To];
10     if(Pos&&Free[Pos].Rest<RootVal[To]){
11         Used[Pos]=true;
12         Need[To]=false;
13     }
14 }
```

正解 100 Pts

```
1  for(int i=Head[1];i;i=Edge[i].Next){
2      int To=Edge[i].Ed;
3      if(Need[To]) NeedVal[++cn]=RootVal[To];
4  }
5  for(int i=1;i<=cf;i++){
6      if(!Used[i]) RestList[++cr]=Free[i].Rest;
7  }
8  sort(NeedVal+1,NeedVal+cn+1);
9  sort(RestList+1,RestList+cr+1);
10 int Pos=1;
11 for(int i=1;i<=cr&&Pos<=cn;i++){
12     if(RestList[i]>=NeedVal[Pos]) Pos++;
13 }
14 return Pos>cn;
```

Contents

T1 能量获取 (select)

T2 策划道路 (city)

T3 异色飞羽 (beads)

T4 战争游戏 (wargame)

吐槽时间